SOFTWARE ENGINEERING

LAB 6

REPORT

Name	Anushka P
SRN	PES1UG24CS075
Section	B

1. Introduction

This lab focuses on testing the electricity bill application and measuring the extent to which its implementation is exercised by the test suite. The application contains the functions energy_charge(), late_fee(), and subsidy(). The testing work uses pytest together with coverage.py to evaluate statement coverage and branch coverage.

2. Testing Scope and Approach

The test cases exercise the application's main calculation paths and decision conditions. The cases shown in the test output include domestic and commercial connections; energy-unit boundaries at 100 and 300 units; invalid connections and negative units; late-payment boundaries including 15 days and bill amounts of 2,000; and subsidy combinations involving senior-citizen status and solar availability. Boundary values and alternate conditions are included to exercise different paths through the functions.

3. Code Coverage Analysis and Report

The code-coverage test run collected 24 test cases, and all 24 passed. The coverage report shows 32 executable statements in electricity_bill.py, with 0 missed statements and 100% statement coverage. Statement coverage measures whether each executable line of code was run at least once during the test execution. In this run, every statement measured by coverage.py was executed by the test suite.

The test cases cover the main calculation paths in the electricity bill program. For energy_charge(), they check domestic and commercial connections, zero and low usage, the 100-unit and 300-unit boundaries, usage above 300 units, and invalid inputs such as an unknown connection or negative units. For late_fee(), they test on-time payment, negative or zero days late, the 15-day boundary, delays beyond 15 days, and bill amounts at and above 2,000. For subsidy(), they test senior and non-senior customers, solar and non-solar cases, and usage above 300 units. These cases ensure that all statements within the three functions are reached during execution.

The result of 100% statement coverage means that no executable statement was left untested in this run. It is a strong indication that the test suite has exercised the complete set of measured statements. However, statement coverage alone does not prove that every possible input or every decision outcome is correct; that is why branch coverage is analysed separately.

Command used

Coverage interpretation: 32 statements executed out of 32 measured (32 − 0 missed) = 100% statement coverage.

```
coverage run --source=electricity_bill -m pytest -v CodeCoverageTest_electricity_bill.py
coverage report -m
```

Code coverage terminal output

```
C:\Users\Admin\L6_L7_17_electricity_bill\Lab 6>coverage erase

C:\Users\Admin\L6_L7_17_electricity_bill\Lab 6>coverage run --source=electricity_bill -m pytest -v CodeCoverageTest_electricity_bill.py
========================== test session starts ==========================
platform win32 -- Python 3.12.4, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Admin\L6_L7_17_electricity_bill\Lab 6
collected 24 items

CodeCoverageTest_electricity_bill.py::test_domestic_low_usage PASSED [  4%]
CodeCoverageTest_electricity_bill.py::test_commercial_low_usage PASSED [  8%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_exactly_100_units PASSED [ 12%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_above_100_units PASSED [ 16%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_exactly_300_units PASSED [ 20%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_above_300_units PASSED [ 25%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_zero_units PASSED [ 29%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_invalid_connection PASSED [ 33%]
CodeCoverageTest_electricity_bill.py::test_energy_charge_negative_units PASSED [ 37%]
CodeCoverageTest_electricity_bill.py::test_late_fee_zero_days PASSED [ 41%]
CodeCoverageTest_electricity_bill.py::test_late_fee_negative_days PASSED [ 45%]
CodeCoverageTest_electricity_bill.py::test_late_fee_one_day PASSED   [ 50%]
CodeCoverageTest_electricity_bill.py::test_late_fee_exactly_15_days PASSED [ 54%]
CodeCoverageTest_electricity_bill.py::test_late_fee_more_than_15_days PASSED [ 58%]
CodeCoverageTest_electricity_bill.py::test_late_fee_bill_exactly_2000 PASSED [ 62%]
CodeCoverageTest_electricity_bill.py::test_late_fee_bill_above_2000 PASSED [ 66%]
CodeCoverageTest_electricity_bill.py::test_senior_citizen_subsidy PASSED [ 70%]
CodeCoverageTest_electricity_bill.py::test_non_senior_citizen PASSED [ 75%]
CodeCoverageTest_electricity_bill.py::test_senior_citizen_above_100_units PASSED [ 79%]
CodeCoverageTest_electricity_bill.py::test_solar_subsidy PASSED      [ 83%]
CodeCoverageTest_electricity_bill.py::test_senior_with_solar_subsidy PASSED [ 87%]
CodeCoverageTest_electricity_bill.py::test_solar_subsidy_above_300_units PASSED [ 91%]
CodeCoverageTest_electricity_bill.py::test_senior_with_solar_above_300_units PASSED [ 95%]
CodeCoverageTest_electricity_bill.py::test_no_subsidy PASSED         [100%]

========================== 24 passed in 0.19s ===========================

C:\Users\Admin\L6_L7_17_electricity_bill\Lab 6>coverage report -m
Name                  Stmts   Miss  Cover   Missing
---------------------------------------------------
electricity_bill.py      32      0   100%
---------------------------------------------------
TOTAL                    32      0   100%

C:\Users\Admin\L6_L7_17_electricity_bill\Lab 6>
```

Figure 1. Pytest execution and statement coverage report showing 24 passed tests and 100% coverage.

4. Branch Coverage Analysis and Report

Branch coverage measures whether the possible outcomes of decision points in the program have been executed. A decision such as an if/else condition can lead to different paths depending on the input. Branch testing therefore checks that the relevant outcomes of these decisions are exercised, rather than only confirming that the line containing the decision was executed.

The branch-coverage test run collected 27 test cases, and all 27 passed. The coverage report records 32 statements, 0 missed statements, 22 branches, 0 partial branches, and 100% coverage for electricity_bill.py. The 0 partial branches result indicates that every branch outcome measured by coverage.py was exercised by the tests in this run.

The branch-focused test cases check both sides of the important conditions in the application. These include valid and invalid connection types; negative and non-negative units; energy usage at, below, and above the 100-unit and 300-unit limits; payment delays of zero, within 15 days, exactly 15 days, and beyond 15 days; bill amounts below, exactly at, and above 2,000; senior-citizen eligibility; and solar-subsidy conditions, including usage above 300 units. Testing these alternatives helps verify that the program takes the intended paths when the input changes.

The 100% branch coverage result, together with 0 partial branches, shows that all measured decision outcomes were covered. This complements the 100% statement coverage result: statement coverage confirms that all measured executable statements ran, while branch coverage provides additional evidence that the alternative outcomes of the decisions were also exercised.

The branch-focused cases cover both sides of important conditions, including valid and invalid connection types, negative and non-negative units, the energy-charge boundaries, late-payment day ranges, the bill-amount threshold, senior-citizen eligibility, and solar-subsidy conditions. The report's 0 partial branches indicates that all measured branch outcomes were covered by the executed tests.

Command used

Coverage interpretation: 0 missed statements and 0 partial branches were reported; all 22 measured branches were covered, resulting in 100% reported coverage.

```
coverage run --branch --source=electricity_bill -m pytest -v BranchCoverageTest_electricity_bill.py
coverage report -m
```

Branch coverage terminal output

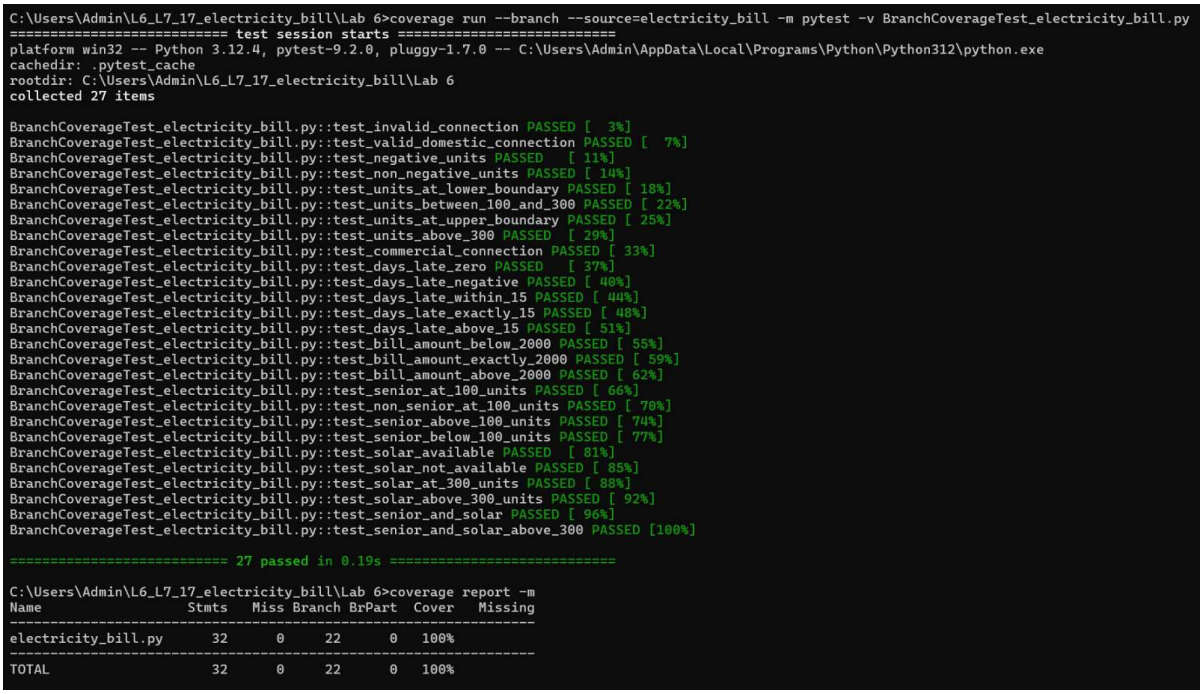


Figure 2. Pytest execution and branch coverage report showing 27 passed tests, 22 branches, 0 partial branches, and 100% coverage.

5. Summary of Results

Measurement	Test cases passed	Coverage result
Code coverage	24 / 24	100% statement coverage
Branch coverage	27 / 27	100% coverage; 0 partial branches

6. Conclusion

Both test suites completed successfully, with no test failures. The code-coverage run achieved 100% statement coverage across 32 statements, with 0 missed statements. The branch-coverage run also achieved 100% coverage, with 22 branches and 0 partial branches. These results show that the test cases executed every statement and every measured branch outcome reported for electricity_bill.py.

Using both measures gives a more complete view of the testing performed. Statement coverage confirms that the program's executable statements were reached, while branch coverage confirms that the alternative outcomes of its decision conditions were exercised. The successful results demonstrate that the test suite provides thorough coverage of the electricity bill application's implemented logic. Coverage is a measure of test execution, not a guarantee that the program is free of all defects; the expected results of the individual test cases must also be checked.

The report is based on the terminal outputs captured for this lab. The test counts and coverage figures stated above are taken directly from those outputs.